algorithm comparism

July 4, 2021

The algorithm of Reversible Data Hiding in Encrypted Images Based on Multi-MSB Prediction and Huffman Coding, a.k.a. msb_huffman.

Our algorithm, a.k.a. shorter_huffman.

1 msb_huffman embedding

1. Generate Label Map.

   INPUT: Cover image

   OUTPUT: Label map, frequency table

   (a) Skip first row and first column since they are defined as Reference Values.

   (b) For the rest of the pixels of cover image

   i. $pv(i,j) = \begin{cases} max(x_2, x_3) & , x_1 \leq min(x_2, x_3) \\ min(x_2, x_3) & , x_1 \geq max(x_2, x_3) \\ x_2 + x_3 - x_1 & , otherwise \end{cases}$

   ii. $idx =$ the index of the MSB of $pv(i,j) \oplus pixel(i,j)$ counting from LSB.

   iii. $t = \begin{cases} 8 & , idx = 0 \\ 8 - idx + 1 & , otherwise \end{cases}$

   iv. calculate $freq[t]$.

2. Generate Huffman coding rule from a fixed Huffman tree.

   INPUT: frequency table

   OUTPUT: Huffman coding rule, capacity

   (a) Skip first row and first column.

   (b) For the rest of the label map, calculate the $freq$ of every t.

   (c) Sort t by $freq$.

   (d) Map each t to a fixed Huffman coding rule {00, 01, 100, 101, 1100, 1101, 1110, 11110, 11111}.

(e) For each t in frequency table, calculate

i. $LM = \sum_{t=0}^{8}(freq[t] * \text{the Huffman code length of } t)$

ii. $cap = \sum_{t=0}^{8}(freq[t] * c[t]), c[t] = \begin{cases} t & , t = 8 \\ t+1 & , otherwise \end{cases}$

(f) the total *capacity* is $cap - LM - header$, the *header* is described in next part.

3. Encode label map

   INPUT: Label map, Huffman coding rule

   OUTPUT: encoded label map.

4. Encrypt the cover image.

   INPUT: Cover image, key stream

   OUTPUT: Encrypted image

5. Generate Side information

   INPUT: Label map, Huffman coding rule, encrypted reference bytes

   OUTPUT: Side information

   (a) the first 32 bits are the Huffman coding rule sorted by t.

   (b) the following 20 bits are the value of LM.

   (c) the LM bits Huffman encoded label map follows.

   (d) the encrypted first row and first column reference values are appended afterward.

6. Replace the encrypted reference byte with side information(part 1).

   INPUT: encrypted image, side information

   OUTPUT: stego image(part 1)

7. The rest of the side information(part 2) is embed into the $t + 1$ MSB of each encrypted pixel.

   INPUT: stego image(part 1)

   OUTPUT: stego image(part 2)

8. the rest of the *capacity* is used to embed secret bits.

   INPUT: stego image(part 2)

   OUTPUT: stego image

2 msb_huffman extracting

1. Extract side information(part 1) from the first row and first column of the stego image.

   INPUT: stego image

   OUTPUT: side information(part 1), Huffman coding rule, label map(part 1)

   (a) Extract the Huffman coding rules from the first 32 bits of the extracted side information.

   (b) Extract LM from the following 20 bits.

   (c) For the rest of the extracted side information(part 1), decode by Huffman coding rules and re-construct as many t of label map as possible.

2. According to the already re-constructed part of the label map, keep extract side information(part 2).

   INPUT: stego image, Huffman coding rule, label map(part 1)

   OUTPUT: side information(part 2), label map.

   For every encrypted pixel except the reference bytes, calculate

   (a) Extract the $t+1$ MSB from the encrypted $pixel(i,j)$ and put them in a $buffer$.

      i. if the $buffer$ become a valid Huffman code, then decode it into the next t.

   (b) This process stops when all the t of the lable map has been decoded.

3. Extract the encrypted reference values and restore them into the stego image.

   INPUT: stego image, label map

   OUTPUT: encrypted image(part 1).

4. Extract secret bits.

   INPUT: stego image, label map

   OUTPUT: secret bits.

5. Restore the encrypted image.

   INPUT: encrypted image(part 1), label map

   OUTPUT: Encrypted image.

For every encrypted pixel except the reference bytes, calculate

(a) $pv(i,j) = \begin{cases} max(x_2, x_3) & , x_1 \leq min(x_2, x_3) \\ min(x_2, x_3) & , x_1 \geq max(x_2, x_3) \\ x_2 + x_3 - x_1 & , otherwise \end{cases}$

(b) replace the $t+1$ MSB of encrypted $pixel(i,j)$ with the $t+1$ MSB of $pv(i,j)$

6. Decrypt the cover image.

   INPUT: Encrypted image, key stream

   OUTPUT: Cover image

3 shorter_huffman embedding

1. Generate Label Map.

   INPUT: Cover image

   OUTPUT: Label map, frequency table

   (a) Skip first 16 bytes since they are defined as Reference Values.

   (b) For the rest of the pixels of the first row, $pv(0,j) = pixel(0,j-1) \oplus pixel(0,j)$.

   (c) For the first pixel of each row, except the first pixel of the first row, $pv(i,0) = pixel(i-1,0) \oplus pixel(i,0)$.

   (d) For the rest of the pixels of cover image, $pv(i,j) = \begin{cases} max(x_2, x_3) & , x_1 \leq min(x_2, x_3) \\ min(x_2, x_3) & , x_1 \geq max(x_2, x_3) \\ x_2 + x_3 - x_1 & , otherwise \end{cases}$

   (e) For all the pixels except reference values,

      i. idx = the index of the MSB of $pv(i,j) \oplus pixel(i,j)$ counting from LSB.

      ii. $t = \begin{cases} 8 & , idx = 0 \\ 8 - idx + 1 & , otherwise \end{cases}$

      iii. calculate $freq[t]$.

2. Generate Huffman tree.

   INPUT: Label map, frequency table

   OUTPUT: Huffman coding rule, capacity

   (a) Generate a Huffman tree node for each t and $freq[t]$ pair.

   (b) while there are more then 1 such Huffman tree node.

      i. find the 2 Huffman tree node with the lowest frequency.

ii. Generate a new Huffman tree node and set the 2 tree nodes as the left and right sub tree.

iii. set the frequency of the new tree node as the sum of the 2 tree nodes.

iv. remove the 2 tree nodes.

(c) map each t with the corresponding Huffman code.

(d) For each t in frequency table, calculate

i. $LM = \sum_{t=0}^{8}(freq[t] * \text{the Huffman code length of } t)$

ii. $cap = \sum_{t=0}^{8}(freq[t] * c[t]), c[t] = \begin{cases} t & , t = 8 \\ t+1 & , otherwise \end{cases}$

(e) the total *capacity* is $cap - LM - header$, the *header* is described in next part.

3. Encode label map.

   INPUT: Label map, Huffman coding rule

   OUTPUT: encoded label map

4. Encrypt the cover image.

   INPUT: Cover image, key stream

   OUTPUT: Encrypted image

5. Generate Side information

   INPUT: Label map, Huffman coding rule, encrypted reference bytes

   OUTPUT: Side information

   (a) the first 17 bits are the Huffman coding tree topology.

   (b) the following 36 bits are the Huffman coding value.

   (c) the following 20 bits are the value of LM.

   (d) the LM bits Huffman encoded label map follows.

   (e) the first 16 encrypted reference bytes are appended afterward.

6. Replace the 16 encrypted reference byte with side information(part 1).

   INPUT: encrypted image, side information

   OUTPUT: stego image(part 1)

7. The rest of the side information(part 2) is embed into the $t + 1$ MSB of each encrypted pixel.

   INPUT: stego image(part 1)

OUTPUT: stego image(part 2)

8. the rest of the *capacity* is used to embed secret bits.

   INPUT: stego image(part 2)

   OUTPUT: stego image

4 shorter_huffman extracting

1. Extract side information(part 1) from the first 16 bytes of the stego image.

   INPUT: stego image

   OUTPUT: side information(part 1), Huffman coding rule, label map(part 1)

   (a) Extract the topology of Huffman coding tree from the first 17 bits of the extracted side information.

   (b) Extract the following 36 bits as 9 Huffman coding values, 4 bits each.

   (c) Extract LM from the following 20 bits.

   (d) For the rest of the extracted side inforation(part 1), decode by Huffman coding rules and re-construct as many t of label map as possible.

2. According to the already re-constructed part of the label map, keep extract side information(part 2).

   INPUT: stego image, Huffman coding rule, label map(part 1)

   OUTPUT: side information(part 2), label map.

   For every encrypted pixel except the reference bytes, calculate

   (a) Extract the $t+1$ MSB from the encrypted $pixel(i,j)$ and put them in a $buffer$.

      i. if the $buffer$ become a valid Huffman code, then decode it into the next t.

   (b) This process stops when all the t of the lable map has been decoded.

3. Extract the encrypted reference values and restore them into the stego image.

   INPUT: stego image, label map

   OUTPUT: encypted image(part 1).

4. Extract secret bits.

   INPUT: stego image, label map

Table 1: msb_huffman with Lena

t	dist	code	cap.	c len	p len	delta
0	9818	1E	1	5	-4	-39272
1	9742	1F	2	5	-3	-29226
2	15247	E	3	4	-1	-15247
3	33246	5	4	3	1	33246
4	44509	1	5	2	3	133527
5	53359	0	6	2	4	213436
6	41758	4	7	3	4	167032
7	24353	D	8	4	4	97412
8	29089	C	8	4	4	116356

OUTPUT: secret bits.

5. Restore the encrypted image.

   INPUT: encrypted image(part 1), label map

   OUTPUT: Encrypted image.

   For every encrypted pixel except the reference bytes, calculate

   (a) for the pixel in the first row except reference bytes, $pv(0,j) = pixel(0,j-1) \oplus pixel(0,j)$.

   (b) For the first pixel of each row, except the first pixel of the first row, $pv(i,0) = pixel(i-1,0) \oplus pixel(i,0)$.

   (c) $pv(i,j) = \begin{cases} max(x_2,x_3) & ,x_1 \leq min(x_2,x_3) \\ min(x_2,x_3) & ,x_1 \geq max(x_2,x_3) \\ x_2+x_3-x_1 & ,otherwise \end{cases}$

   (d) replace the $t+1$ MSB of encrypted $pixel(i,j)$ with the $t+1$ MSB of $pv(i,j)$

6. Decrypt the cover image.

   INPUT: Encrypted image, key stream

   OUTPUT: Cover image

Table 2: shorter_huffman with Lena

t	freq	code	cap.	c len	p len	delta
0	9867	10111	1	5	-4	-39468
1	9751	10110	2	5	-3	-29253
2	15321	1010	3	4	-1	-15321
3	33303	100	4	3	1	33303
4	44679	111	5	3	2	89358
5	53604	00	6	2	4	214416
6	41938	110	7	3	4	167752
7	24452	010	8	3	5	122260
8	29203	011	8	3	5	146015